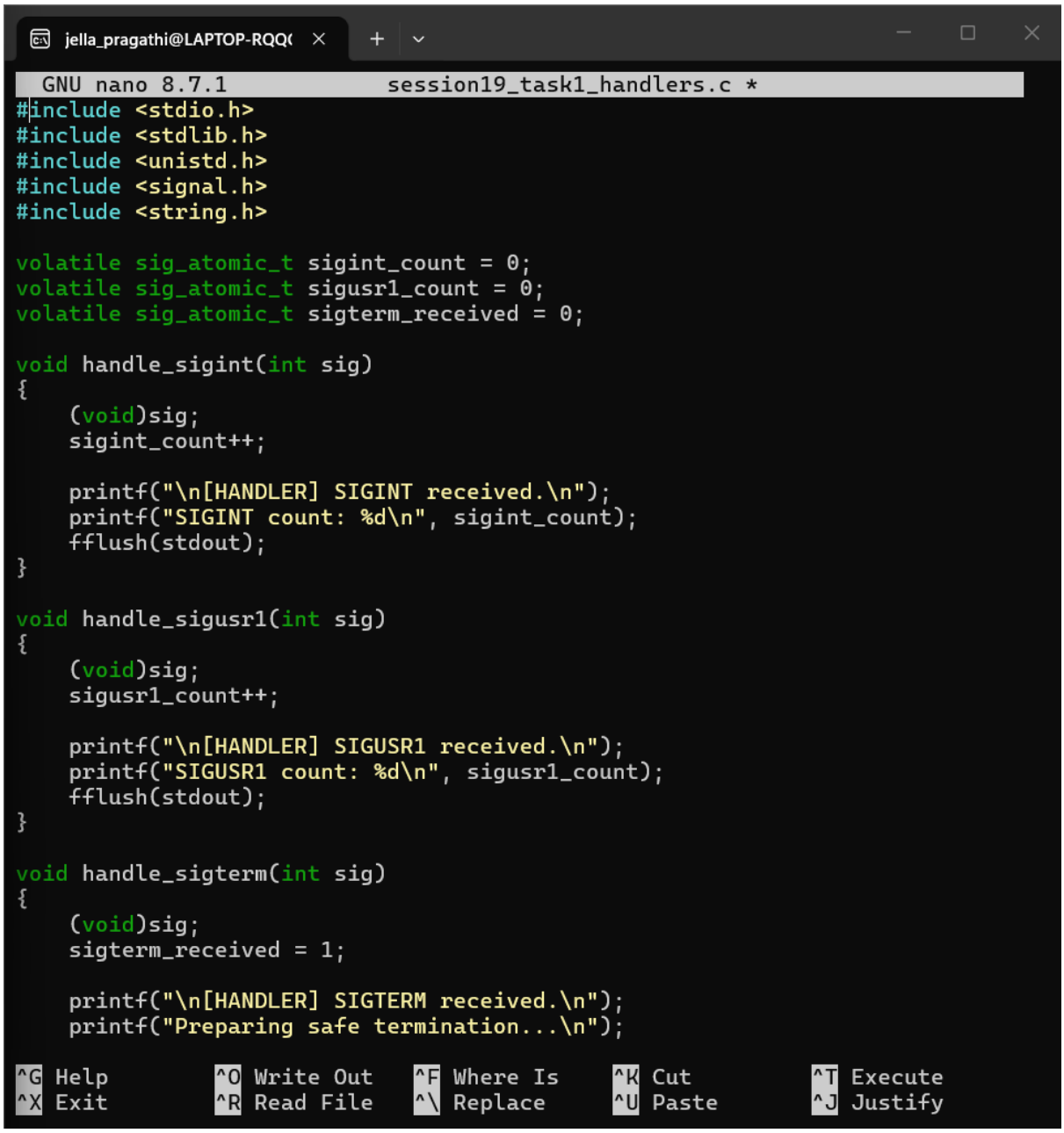
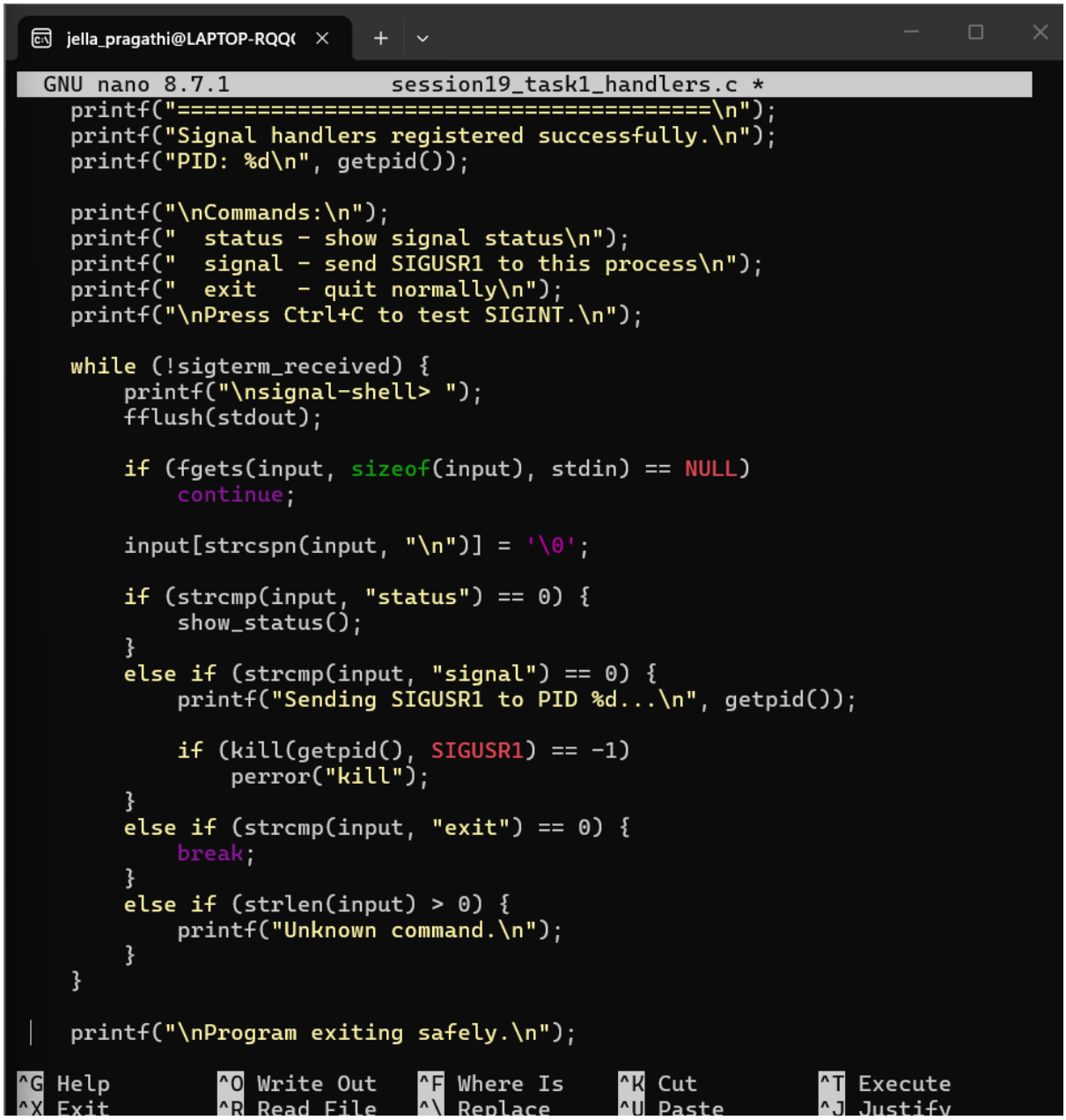
Skill Session 19

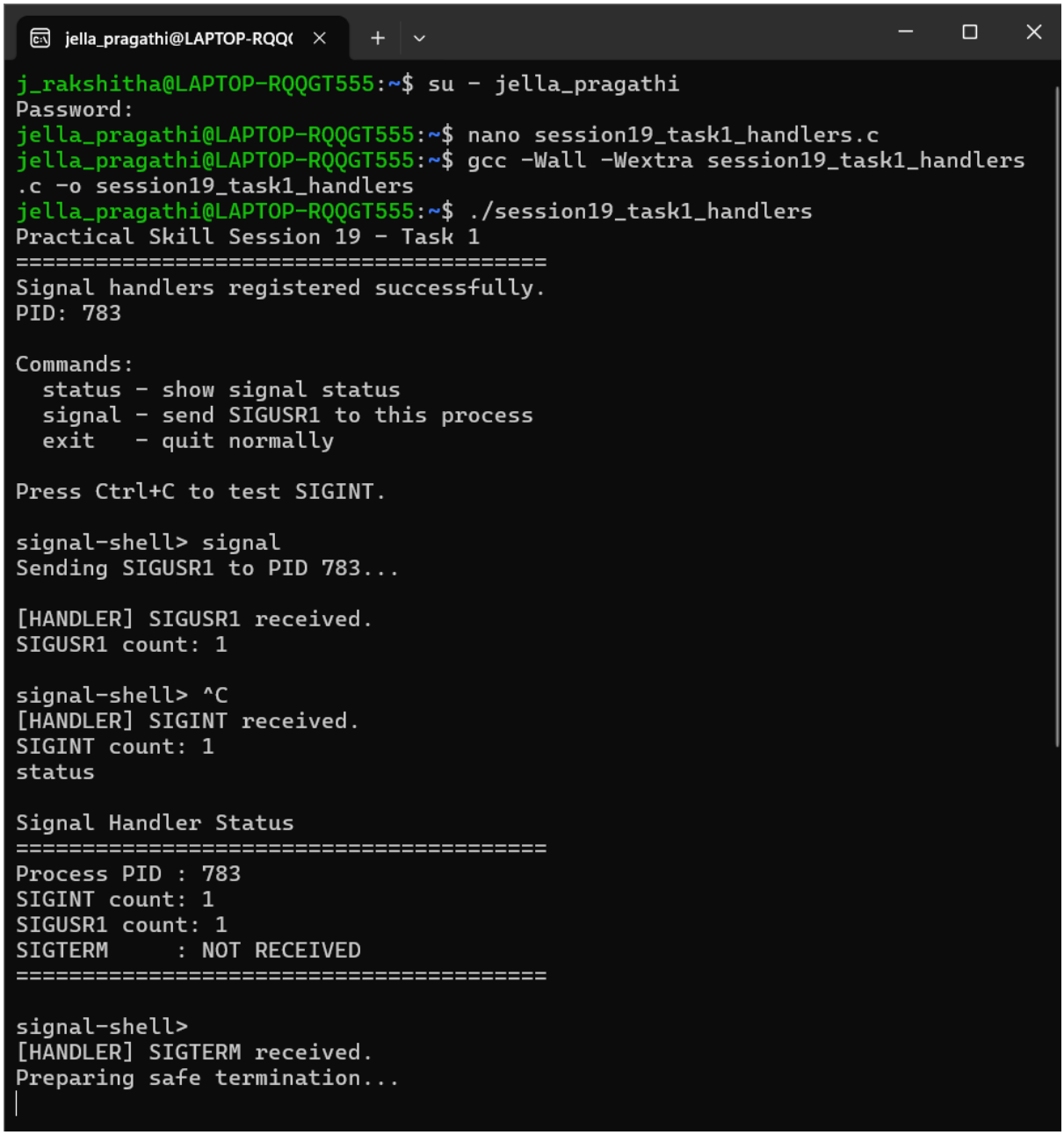
Task 1

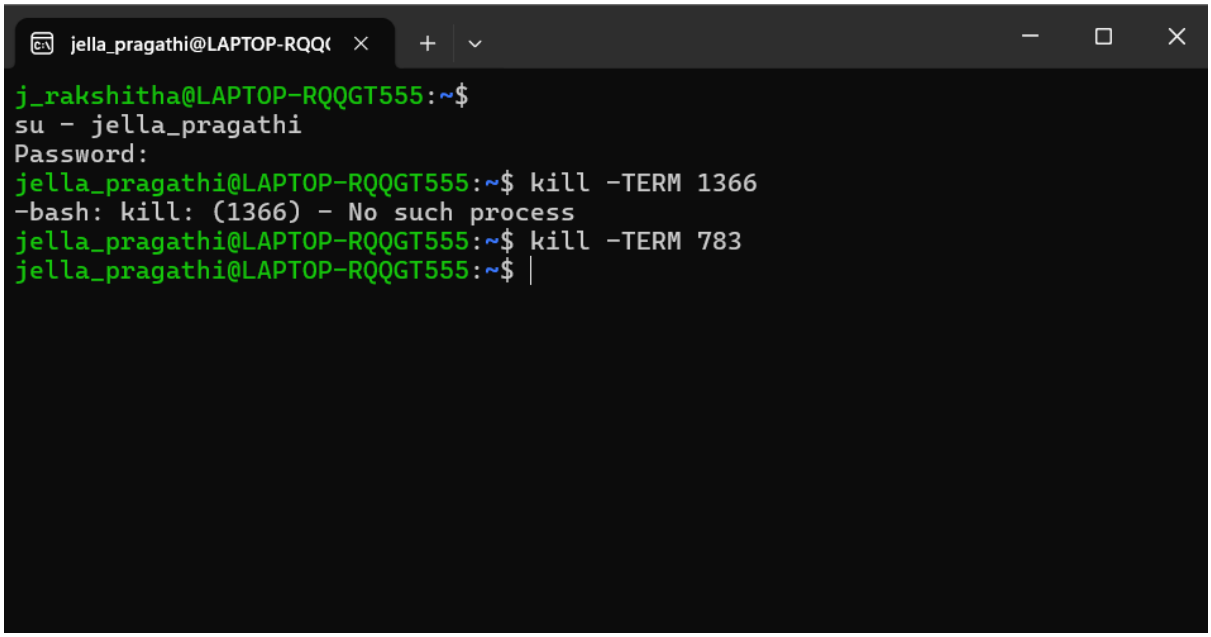
To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events




jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session19_task1_handlers.c *
fflush(stdout);
}
void install_handler(int signal_number, void (*handler)(int))
{
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sigemptyset(&sa.sa_mask);
sa.sa_handler = handler;
sa.sa_flags = SA_RESTART;
if (sigaction(signal_number, &sa, NULL) == -1) {
perror("sigaction");
exit(EXIT_FAILURE);
}
}
void show_status(void)
{
printf("\nSignal Handler Status\n");
printf("=====================================\n");
printf("Process PID : %d\n", getpid());
printf("SIGINT count: %d\n", sigint_count);
printf("SIGUSR1 count: %d\n", sigusr1_count);
printf("SIGTERM : %s\n",
sigterm_received ? "RECEIVED" : "NOT RECEIVED");
printf("=====================================\n");
}
int main(void)
{
char input[100];
install_handler(SIGINT, handle_sigint);
install_handler(SIGUSR1, handle_sigusr1);
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

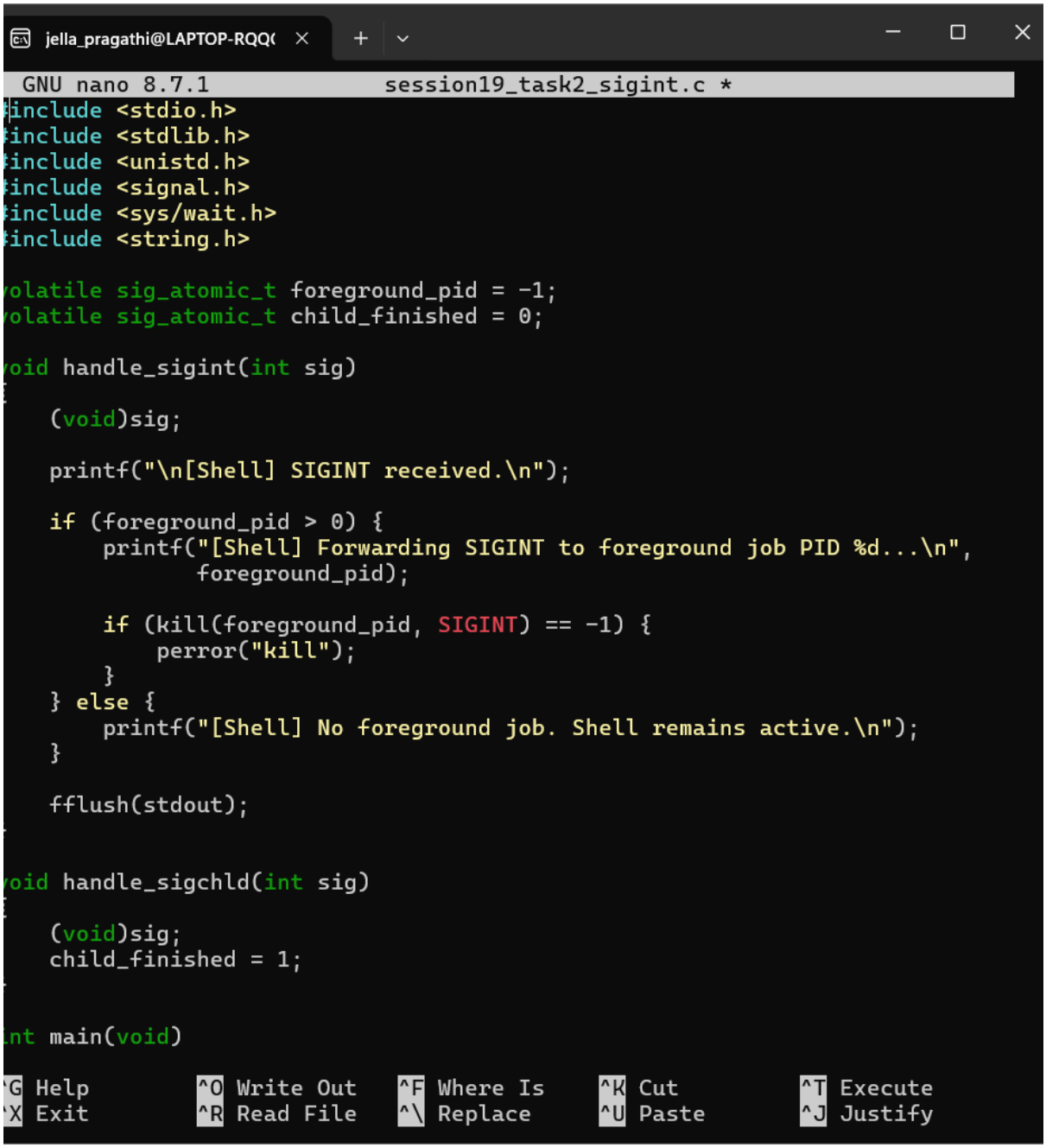

jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session19_task1_handlers.c *
printf("==\n");
printf("Signal handlers registered successfully.\n");
printf("PID: %d\n", getpid());
printf("\nCommands:\n");
printf(" status - show signal status\n");
printf(" signal - send SIGUSR1 to this process\n");
printf(" exit - quit normally\n");
printf("\nPress Ctrl+C to test SIGINT.\n");
while (!sigterm_received) {
printf("\nsignal-shell> ");
fflush(stdout);
if (fgets(input, sizeof(input), stdin) == NULL)
continue;
input[strcspn(input, "\n")] = '\0';
if (strcmp(input, "status") == 0) {
show_status();
}
else if (strcmp(input, "signal") == 0) {
printf("Sending SIGUSR1 to PID %d...\n", getpid());
if (kill(getpid(), SIGUSR1) == -1)
perror("kill");
}
else if (strcmp(input, "exit") == 0) {
break;
}
else if (strlen(input) > 0) {
printf("Unknown command.\n");
}
}
printf("\nProgram exiting safely.\n");
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


jella_pragathi@LAPTOP-RQQ(
j_rakshitha@LAPTOP-RQQGT555:~$ su - jella_pragathi
Password:
jella_pragathi@LAPTOP-RQQGT555:~$ nano session19_task1_handlers.c
jella_pragathi@LAPTOP-RQQGT555:~$ gcc -Wall -Wextra session19_task1_handlers
.c -o session19_task1_handlers
jella_pragathi@LAPTOP-RQQGT555:~$ ./session19_task1_handlers
Practical Skill Session 19 - Task 1
==
Signal handlers registered successfully.
PID: 783
Commands:
status - show signal status
signal - send SIGUSR1 to this process
exit - quit normally
Press Ctrl+C to test SIGINT.
signal-shell> signal
Sending SIGUSR1 to PID 783...
[HANDLER] SIGUSR1 received.
SIGUSR1 count: 1
signal-shell> ^C
[HANDLER] SIGINT received.
SIGINT count: 1
status
Signal Handler Status
==
Process PID : 783
SIGINT count: 1
SIGUSR1 count: 1
SIGTERM : NOT RECEIVED
==
signal-shell>
[HANDLER] SIGTERM received.
Preparing safe termination...


jella_pragathi@LAPTOP-RQQ(
j_rakshitha@LAPTOP-RQQGT555:~$
su - jella_pragathi
Password:
jella_pragathi@LAPTOP-RQQGT555:~$ kill -TERM 1366
-bash: kill: (1366) - No such process
jella_pragathi@LAPTOP-RQQGT555:~$ kill -TERM 783
jella_pragathi@LAPTOP-RQQGT555:~$

Task 2

To Copy SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling




jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session19_task2_sigint.c *
int main(void)
{
struct sigaction sa_int;
struct sigaction sa_chld;
char input[100];
memset(&sa_int, 0, sizeof(sa_int));
sigemptyset(&sa_int.sa_mask);
sa_int.sa_handler = handle_sigint;
sa_int.sa_flags = SA_RESTART;
if (sigaction(SIGINT, &sa_int, NULL) == -1) {
perror("sigaction SIGINT");
return 1;
}
memset(&sa_chld, 0, sizeof(sa_chld));
sigemptyset(&sa_chld.sa_mask);
sa_chld.sa_handler = handle_sigchld;
sa_chld.sa_flags = SA_RESTART;
if (sigaction(SIGCHLD, &sa_chld, NULL) == -1) {
perror("sigaction SIGCHLD");
return 1;
}
printf("Practical Skill Session 19 - Task 2\n");
printf("===\n");
printf("SIGINT forwarding shell started.\n");
printf("Shell PID: %d\n", getpid());
printf("\nCommands:\n");
printf(" run <seconds> - start a foreground job\n");
printf(" status - show job state\n");
printf(" exit - quit shell\n");
printf("\nPress Ctrl+C while a job is running to forward SIGINT.\n");
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session19_task2_sigint.c *
while (1) {
printf("\nsigint-shell> ");
fflush(stdout);
if (fgets(input, sizeof(input), stdin) == NULL)
break;
if (strncmp(input, "run ", 4) == 0) {
int seconds;
if (sscanf(input + 4, "%d", &seconds) != 1 ||
seconds <= 0) {
printf("Usage: run <seconds>\n");
continue;
}
pid_t pid = fork();
if (pid < 0) {
perror("fork");
continue;
}
if (pid == 0) {
signal(SIGINT, SIG_DFL);
printf("[Job] PID %d started for %d seconds.\n",
getpid(), seconds);
fflush(stdout);
for (int i = 1; i <= seconds; i++) {
printf("[Job] PID %d working... %d/%d\n",
getpid(), i, seconds);
fflush(stdout);
sleep(1);
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session19_task2_sigint.c *
printf("[Job] PID %d completed normally.\n", getpid());
fflush(stdout);
exit(0);
}
foreground_pid = pid;
child_finished = 0;
printf("[Shell] Foreground job started: PID %d\n", pid);
int status;
while (!child_finished) {
pid_t result = waitpid(pid, &status, WNOHANG);
if (result == pid) {
child_finished = 1;
} else if (result == -1) {
perror("waitpid");
child_finished = 1;
} else {
sleep(1);
}
}
if (WIFSIGNALED(status)) {
printf("[Shell] Foreground job PID %d terminated by signal >
pid, WTERMSIG(status));
} else if (WIFEXITED(status)) {
printf("[Shell] Foreground job PID %d exited with status %d>
pid, WEXITSTATUS(status));
}
foreground_pid = -1;
printf("[Shell] Foreground job state updated: COMPLETED/TERMINA>
}
else if (strcmp(input, "status\n") == 0) {
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```
        else if (strcmp(input, "status\n") == 0) {
            if (foreground_pid > 0)
                printf("[Shell] Foreground job PID %d is active.\n",
                       foreground_pid);
            else
                printf("[Shell] No active foreground job.\n");
        }
        else if (strcmp(input, "exit\n") == 0) {
            break;
        }
        else if (strlen(input) > 1) {
            printf("Unknown command.\n");
        }
    }

    printf("\nShell exiting safely.\n");
    return 0;
}
```

```
^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify
```

```
jella_pragathi@LAPTOP-RQQGT555:~$ nano session19_task2_sigint.c
jella_pragathi@LAPTOP-RQQGT555:~$ gcc -Wall -Wextra session19_task2_sigint.c
 -o session19_task2_sigint
jella_pragathi@LAPTOP-RQQGT555:~$ ./session19_task2_sigint
Practical Skill Session 19 - Task 2
=====================================
SIGINT forwarding shell started.
Shell PID: 1102

Commands:
  run <seconds> - start a foreground job
  status        - show job state
  exit          - quit shell

Press Ctrl+C while a job is running to forward SIGINT.

sigint-shell> run 10
[Shell] Foreground job started: PID 1105
[Job] PID 1105 started for 10 seconds.
[Job] PID 1105 working... 1/10
[Job] PID 1105 working... 2/10
[Job] PID 1105 working... 3/10
[Job] PID 1105 working... 4/10
[Job] PID 1105 working... 5/10
[Job] PID 1105 working... 6/10
[Job] PID 1105 working... 7/10
[Job] PID 1105 working... 8/10
[Job] PID 1105 working... 9/10
[Job] PID 1105 working... 10/10
[Job] PID 1105 completed normally.
[Shell] Foreground job PID 1105 exited with status 0.
[Shell] Foreground job state updated: COMPLETED/TERMINATED

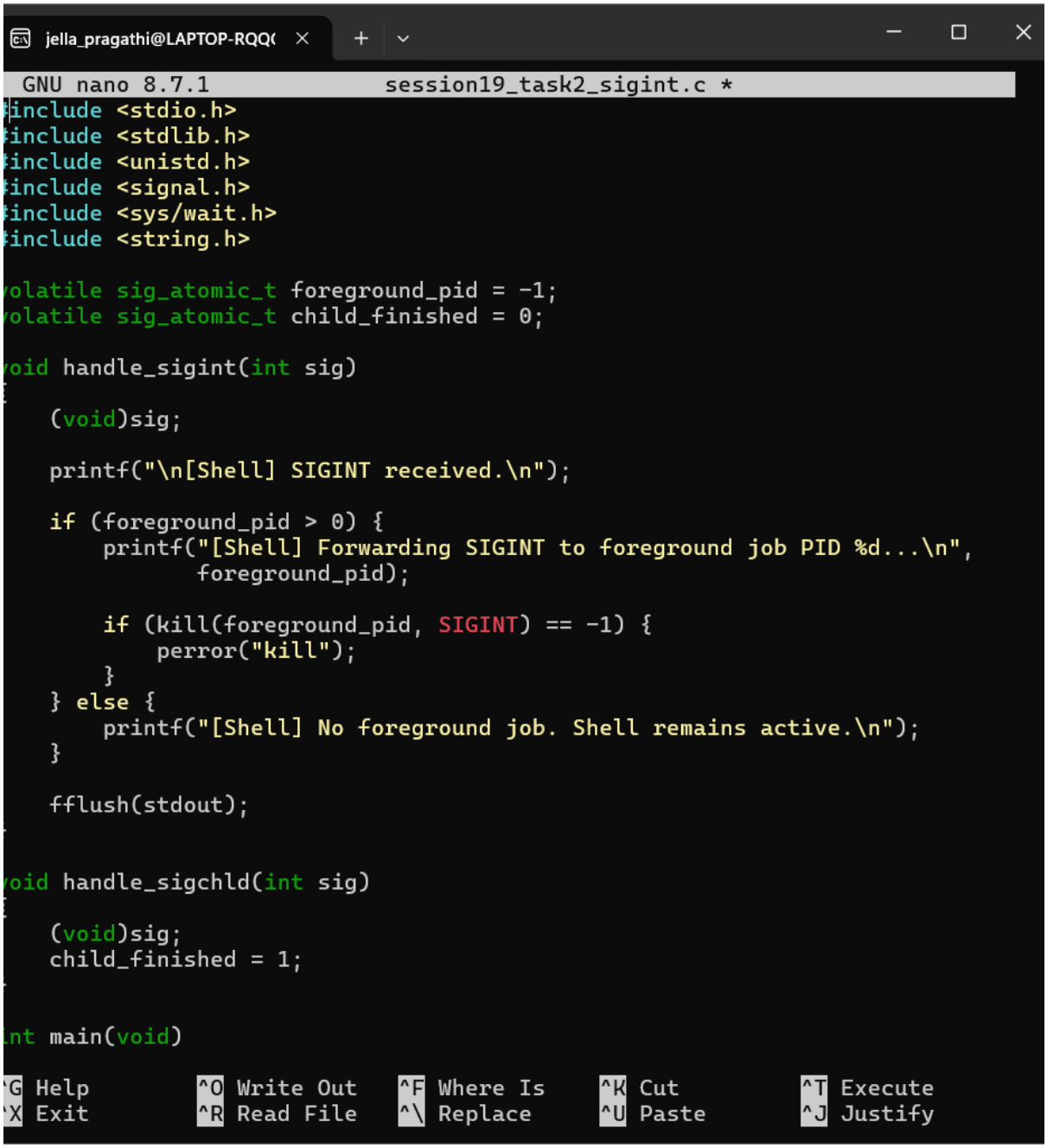
sigint-shell> exit

Shell exiting safely.
```

Skill Session 20

Task 1

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow




jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session19_task2_sigint.c *
int main(void)
{
struct sigaction sa_int;
struct sigaction sa_chld;
char input[100];
memset(&sa_int, 0, sizeof(sa_int));
sigemptyset(&sa_int.sa_mask);
sa_int.sa_handler = handle_sigint;
sa_int.sa_flags = SA_RESTART;
if (sigaction(SIGINT, &sa_int, NULL) == -1) {
perror("sigaction SIGINT");
return 1;
}
memset(&sa_chld, 0, sizeof(sa_chld));
sigemptyset(&sa_chld.sa_mask);
sa_chld.sa_handler = handle_sigchld;
sa_chld.sa_flags = SA_RESTART;
if (sigaction(SIGCHLD, &sa_chld, NULL) == -1) {
perror("sigaction SIGCHLD");
return 1;
}
printf("Practical Skill Session 19 - Task 2\n");
printf("===\n");
printf("SIGINT forwarding shell started.\n");
printf("Shell PID: %d\n", getpid());
printf("\nCommands:\n");
printf(" run <seconds> - start a foreground job\n");
printf(" status - show job state\n");
printf(" exit - quit shell\n");
printf("\nPress Ctrl+C while a job is running to forward SIGINT.\n");
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```
jella_pragathi@LAPTOP-RQQGT555:~$ nano session19_task2_sigint.c
jella_pragathi@LAPTOP-RQQGT555:~$ gcc -Wall -Wextra session19_task2_sigint.c
 -o session19_task2_sigint
jella_pragathi@LAPTOP-RQQGT555:~$ ./session19_task2_sigint
Practical Skill Session 19 - Task 2
=====================================
SIGINT forwarding shell started.
Shell PID: 1102

Commands:
  run <seconds> - start a foreground job
  status        - show job state
  exit          - quit shell

Press Ctrl+C while a job is running to forward SIGINT.

sigint-shell> run 10
[Shell] Foreground job started: PID 1105
[Job] PID 1105 started for 10 seconds.
[Job] PID 1105 working... 1/10
[Job] PID 1105 working... 2/10
[Job] PID 1105 working... 3/10
[Job] PID 1105 working... 4/10
[Job] PID 1105 working... 5/10
[Job] PID 1105 working... 6/10
[Job] PID 1105 working... 7/10
[Job] PID 1105 working... 8/10
[Job] PID 1105 working... 9/10
[Job] PID 1105 working... 10/10
[Job] PID 1105 completed normally.
[Shell] Foreground job PID 1105 exited with status 0.
[Shell] Foreground job state updated: COMPLETED/TERMINATED

sigint-shell> exit

Shell exiting safely.
```